

MODULE 21 · DOMAIN 2 - DESIGN FOR NEW SOLUTIONS

Designing for Performance

A hierarchy of caches. Each layer removes work from the one below it.

DOMAIN 2 of 4 EXAM WEIGHT 29% MODULE 21 of 44

The fastest request is the one that **never reaches your origin**. Work backwards from the user.

The hierarchy

01

Edge

CloudFront caches the response near the user.

02

Application

ElastiCache holds computed results and sessions.

03

Database

Read replicas absorb reads the cache missed.

04

Origin

Only genuinely new work arrives here.

ON THE EXAM

Each layer should absorb what the one below it cannot afford. Scaling the bottom layer to fix the top is the classic error.

CloudFront or Global Accelerator?

AMAZON CLOUDFRONT

A cache at the edge

- Caches content near the user
- HTTP and HTTPS, plus gRPC
- WAF attaches to it
- Cuts origin load, not just latency

GLOBAL ACCELERATOR

A shortcut to the backbone

- Caches nothing at all
- TCP and UDP, any protocol
- Two static anycast IPs
- Fast regional failover

THE TIEBREAKER

Cacheable HTTP means CloudFront. Non-HTTP, or a requirement for static IPs, means Global Accelerator.

CloudFront, and what changed

- ◆ **VPC origins** let CloudFront reach an ALB, NLB or instance in a **private subnet**. No public origin at all.
- ◆ That replaces the old pattern of an internet-facing ALB protected by a secret custom header.
- ◆ **Origin Access Control** is the current way to lock an S3 origin to CloudFront. It replaced Origin Access Identity.
- ◆ **Origin Shield** adds a central caching layer, so many edges do not all miss to your origin at once.

ON THE EXAM

Cache policies decide what forms the cache key. Forwarding every header and cookie destroys your hit rate silently.

CloudFront Functions or Lambda@Edge?

CLOUDFRONT FUNCTIONS

Sub-millisecond, at the edge

- Viewer request and response only
- No network access
- Restricted JavaScript, tiny runtime
- Roughly a sixth of the cost

LAMBDA@EDGE

A real runtime, further back

- Viewer and origin events
- Network calls and AWS SDK access
- Node.js or Python, more memory
- Higher latency and cost

THE TIEBREAKER

Needing to call another service forces Lambda@Edge. Header rewrites and redirects at scale belong in CloudFront Functions.

Caching patterns, and how they fail

- ◆ **Lazy loading** caches on a miss. Simple, and the first request after expiry is always slow.
- ◆ **Write-through** updates the cache on write. Data is always fresh, and you cache things nobody reads.
- ◆ **Cache stampede**: a popular key expires and thousands of requests hit the database at once.
- ◆ Fix it with **jittered TTLs**, so keys do not expire together, or by refreshing before expiry.

ON THE EXAM

A TTL is a correctness decision, not a performance one. It says how stale the answer may be, and everything else follows.

Relieving the database

- ◆ **Read replicas** take read traffic off the writer. The application must know to route reads to them.
- ◆ **RDS Proxy** pools connections, which is what Lambda at scale needs. Thousands of functions, a handful of connections.
- ◆ **DAX** is a DynamoDB-only cache giving microsecond reads. It cannot front anything else.
- ◆ **Buffering with a queue** converts a write spike into a backlog, as in module eighteen.

ON THE EXAM

Too many connections and too many queries are different problems. A proxy fixes the first; a cache fixes the second.

Quotas as a design input

- ◆ **Quotas are per account and per Region**, and default values are not sized for your workload.
- ◆ A new Region for disaster recovery starts at defaults, however large the primary has grown.
- ◆ **Service Quotas** shows current values, and CloudWatch can alarm as usage approaches a limit.
- ◆ Some quotas cannot be raised at all. Design around those rather than planning to request an increase.

ON THE EXAM

A quota reached under load looks exactly like a performance problem, and no amount of scaling fixes it.

Reading the scenario

THE SCENARIO SAYS

Global users, cacheable web content

Global users, UDP or non-HTTP, static IPs

Simple rewrite or redirect, huge volume

Edge logic that must call another service

Lambda exhausting database connections

Origin overwhelmed when a cache entry expires

IT IS POINTING AT

CloudFront

Global Accelerator

CloudFront Functions

Lambda@Edge

RDS Proxy

Jittered TTLs, or Origin Shield

ON THE EXAM

Ask which layer should have absorbed the request. The answer is usually one level above where the symptom appears.

Meridian Freight runs a vehicle telemetry endpoint that receives UDP datagrams from 60,000 trucks worldwide. Latency varies badly for vehicles in Asia and South America, and the operations team needs two fixed IP addresses to configure in the vehicle firmware. The payloads are unique per vehicle and cannot be cached.

What should the solutions architect recommend?

- A CloudFront with a custom origin and caching disabled
- B AWS Global Accelerator in front of Network Load Balancers in several Regions
- C Route 53 latency-based routing to regional endpoints
- D An Application Load Balancer with cross-Region support

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A CloudFront, caching off

CloudFront handles HTTP and gRPC, not UDP. Disabling the cache also removes the only thing it was bringing.

C Route 53 latency routing

Directs clients to a nearby Region and leaves the traffic on the public internet. It also gives DNS names, not fixed IPs.

B Global Accelerator with NLBs

Works with UDP, provides two static anycast IPs for the firmware, and moves traffic onto the AWS backbone early.

D ALB cross-Region

An ALB is HTTP-only and Region-scoped. Neither the protocol nor the topology fits.

THE SIGNAL

UDP, uncacheable, and static IPs. Three separate signals, all pointing away from CloudFront.

A web application runs on an Application Load Balancer in private subnets, reached today through a second internet-facing ALB. Security dislikes the public ALB and protects it with a secret custom header that CloudFront adds.

The team asks whether there is now a cleaner option.

What should the solutions architect recommend?

- ☐ A Keep the secret header pattern; it is the standard approach
- ☐ B Use a CloudFront VPC origin pointing directly at the private ALB
- ☐ C Put the ALB behind a NAT gateway
- ☐ D Replace the ALB with an API Gateway private integration

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

 Keep the secret header
Was the standard answer until late 2024, and it depends on a shared secret and an origin that is still publicly reachable.
- B

 CloudFront VPC origin
CloudFront reaches the private ALB directly. No public origin, no shared secret, and nothing else can reach it.
- C

 ALB behind a NAT gateway
A NAT gateway is for outbound traffic from private subnets. It does not make an ALB reachable from outside.
- D

 API Gateway private integration
Replaces a working load balancer with a different service to solve a connectivity problem that has a direct answer.

THE SIGNAL

This answer changed in November 2024. Material written before then teaches option A, and taught it correctly at the time.

A media site must rewrite incoming URLs to a canonical form and redirect legacy paths, on every one of 8 billion monthly requests. The logic depends only on the request path and a small mapping table that changes weekly.

The team currently does this with Lambda@Edge and finds the cost significant.

What should the solutions architect recommend?

- ☐ A Keep Lambda@Edge but increase its memory to reduce duration
- ☐ B Move the logic to CloudFront Functions, with the mapping in CloudFront KeyValueCollection
- ☐ C Move the rewrites to the origin application
- ☐ D Use an S3 redirect rules configuration

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A More memory

Trims duration on a per-invocation cost that is roughly six times the alternative. It optimises the wrong service.

C Rewrites at the origin

Every request now travels to the origin to be told it went to the wrong place. It removes the point of doing this at the edge.

B CloudFront Functions with KeyValueStore

Path-only logic is exactly the Functions use case, and KeyValueStore holds the weekly mapping with no network call and no redeploy.

D S3 redirect rules

Only applies to an S3 website endpoint, and cannot express a mapping table for a media site behind CloudFront.

THE SIGNAL

The logic needs no network call, which is what makes CloudFront Functions viable. KeyValueStore is what lets the table change weekly.

During traffic peaks, an application backed by Aurora begins returning connection errors. Metrics show CPU on the writer at 35% and the connection count at its maximum. The application runs as several hundred concurrent Lambda functions. Query performance itself is fine.

What should the solutions architect recommend?

- ☐ A Add Aurora read replicas and route reads to them
- ☐ B Introduce RDS Proxy between Lambda and Aurora
- ☐ C Scale the writer to a larger instance class
- ☐ D Add an ElastiCache layer in front of Aurora

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

 Read replicas
Spreads queries, and each Lambda still opens its own connection. Replicas add capacity for a problem that is not query volume.
- B

 RDS Proxy
Pools and reuses connections, so hundreds of concurrent functions share a small pool. Purpose-built for serverless against a relational database.
- C

 Larger writer
Would raise the connection limit as a side effect, at considerable cost, while CPU sits at 35%.
- D

 ElastiCache
Reduces query volume, and the scenario says query performance is fine. It is the wrong bottleneck.

THE SIGNAL

CPU at 35% with connections at maximum is the giveaway. The database is not busy; it has simply run out of sockets.

A news site caches article pages in ElastiCache with a one-hour TTL, populated lazily. Every hour, on the hour, the database briefly saturates and response times spike for about forty seconds.

The cache hit rate is otherwise above 98%.

What is happening, and what should change?

- ☐ A A cache stampede on synchronised expiry; add jitter to TTLs and refresh popular keys before expiry
- ☐ B The cache is undersized; move to a larger node type
- ☐ C Increase the TTL to 24 hours
- ☐ D Switch from lazy loading to Memcached

◆ PAUSE AND DECIDE

What is happening, and what should change?

- A

Stampede; jitter the TTLs
Keys written together expire together, so every miss hits the database at once. Jitter spreads expiry across the hour.
- B

Bigger cache node
A 98% hit rate says the cache is comfortably sized. Evictions are not the problem.
- C

24-hour TTL
Moves the stampede to once a day and makes news articles up to a day stale. It trades correctness for a less frequent outage.
- D

Memcached
A different engine with the same expiry behaviour. The stampede is caused by TTL alignment, not by the engine.

THE SIGNAL

On the hour, every hour, for forty seconds is a schedule, not a load pattern. Something is expiring in unison.

A company fails over to its disaster recovery Region during a drill. The CloudFormation deployment stops partway through, reporting that the maximum number of VPCs has been reached and that an EC2 instance type is not available at the requested count.

The primary Region has grown steadily over four years.

What is the underlying issue?

- ☐ A Service quotas are per Region and the DR Region still sits at defaults
- ☐ B The CloudFormation templates are out of date
- ☐ C The DR Region lacks capacity and AWS must add hardware
- ☐ D Organizations policies are blocking resource creation

◆ PAUSE AND DECIDE

What is the underlying issue?

- A

Quotas are per Region

Four years of increases were requested in the primary Region only. The DR Region has never carried load and still holds default limits.
- B

Stale templates

The templates worked; the deployment was refused. A quota error is not a template error.
- C

Regional capacity

Possible in principle, and a quota message names a quota. AWS capacity shortfalls report differently.
- D

SCPs blocking creation

An SCP produces an access denied error naming the action, not a quota-exceeded message.

THE SIGNAL

Quotas grow invisibly in the Region you use. The DR Region only reveals its limits on the day you need it.

What to carry out of this module

- ◆ **Work backwards from the user.** Ask which layer should have absorbed the request before scaling the one that failed.
- ◆ **CloudFront caches; Global Accelerator does not.** Non-HTTP protocols or static IPs force Global Accelerator.
- ◆ **Network access is the edge compute separator.** CloudFront Functions cannot; Lambda@Edge can, at six times the cost.
- ◆ **Connections and queries are different problems.** RDS Proxy fixes one, a cache fixes the other.

ON THE EXAM

Quotas are per Region. A DR Region sits at defaults until the day you need it, which is the day you discover them.